

Non-Functional Test Cases: Security Testing

Library Management System

Test Design Team (Balanced - Coders & Non-Coders)

June 5, 2025

Note: These test cases are designed based on SRS security requirements and cover both user-facing (black-box) and technical (API/server-side) security aspects. Actual results and status are to be filled during test execution.

IV.A Non-Functional Test Cases: Security Testing

Table 1: Security Test Cases

Test Case ID	Test Title	Test Steps	Expected Results
Sub-Section: Access Control & Authorization (RBAC)			
TC_SEC_RBAC_001	Verify Member cannot access Librarian-only URLs	- Login as a 'Member' user. - Attempt to navigate directly to a Librarian-only URL (e.g., '/users', '/dashboard').	- Access is denied. - The system redirects the user to an appropriate page (e.g., login page, member dashboard) and does not display the requested admin page.
TC_SEC_RBAC_002	Verify unauthenticated user cannot access any protected pages	- Ensure no user is logged in (e.g., use a new browser session or clear cookies). - Attempt to navigate directly to any protected URL (e.g., '/books', '/users', '/borrowals').	- Access is denied for all protected URLs. - The system redirects the user to the login page.
TC_SEC_RBAC_003	Member attempts to perform a Librarian action via API	- Login as a 'Member' user and obtain the session token. - Use an API tool (e.g., Postman) to send a request to a Librarian-only API endpoint (e.g., 'DELETE /api/user/delete/:id' with a valid user ID).	- The API returns an authorization error (e.g., 403 Forbidden or 401 Unauthorized). - The action (e.g., deleting a user) is not performed. The data remains unchanged in the database.
Sub-Section: Input Validation & Injection Attacks			
TC_SEC_INJ_001	Attempt basic Cross-Site Scripting (XSS) in UI form fields	- Login as Librarian. - Navigate to the 'Add Book' form. - In a text field (e.g., 'Summary'), enter a basic XSS payload like <code><script>alert('XSS')</script></code>. - Submit the form to create the book. - View the details of the newly created book.	- The script is not executed. No alert box appears. - The input is properly sanitized and displayed as plain text (e.g., <code><script>alert('XSS')</script></code>) or the input is rejected.
TC_SEC_INJ_002	Attempt basic NoSQL injection in login form	- Navigate to the login page. - In the email/username field, enter a NoSQL injection payload (e.g., <code>{ \$ne: null }</code>). - Enter any password and submit.	- The login attempt fails. - The system returns a generic error message like <code>"Invalid username or password"</code> and does not crash or reveal system information. - The application is not bypassed.
Continued on next page			

Table 1 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_INJ_003	Attempt insecure direct object reference (IDOR) via API	• Login as 'Member A'. • Navigate to 'My Borrowal History' and note the URL or API call used to fetch the data (e.g., 'GET /api/borrowal/getAll' which is then filtered by the client). • Using an API tool, attempt to request details of a specific borrowal that belongs to 'Member B' by guessing or obtaining its ID (e.g., 'GET /api/borrowal/get/:borrowal_id_of_member_B').	• The API returns an authorization error (e.g., 403 Forbidden or 404 Not Found). • The details of 'Member B's borrowal record are not displayed.
Sub-Section: Session Management			
TC_SEC_SESS_001	Verify session invalidation on logout	• Login as any user. • Copy the URL of a protected page (e.g., '/books'). • Click the 'Logout' button. The user is redirected to the login page. • Click the browser's 'Back' button or paste the copied URL into the address bar.	• The protected page is not displayed. • The user remains on the login page or is redirected back to it, confirming the session was terminated.
TC_SEC_SESS_002	Verify secure session cookie attributes	• Login to the application. • Use browser developer tools to inspect the cookies set by the application.	• Session cookies should have the 'HttpOnly' attribute to prevent access via JavaScript. • Session cookies should have the 'Secure' attribute (if using HTTPS) to ensure they are sent only over secure connections.
Sub-Section: Data Security & Transport Layer			
TC_SEC_DATA_001	Verify sensitive data is not returned in API responses	• Login as Librarian. • Use an API tool to call an endpoint that returns user data (e.g., 'GET /api/user/get/:id'). • Inspect the JSON response body.	• The response does not contain sensitive data like the user's password hash or salt. • Only necessary and non-sensitive user data is returned (e.g., name, email, role).
TC_SEC_DATA_002	Verify secure data transmission (HTTPS)	• Access the application in a web browser. • Check the URL in the address bar. • Use browser developer tools to monitor network traffic during login and other operations.	• The connection uses HTTPS, indicated by 'https://' and a lock icon. • All data, especially login credentials, is sent over the encrypted HTTPS connection.
Sub-Section: Static Testing (Code Review)			
Continued on next page			

Table 1 – continued from previous page

Test Case ID	Test Title	Test Steps	Expected Results
TC_SEC_STATIC_001	Code review for security vulnerabilities	• (Led by Coders) • Systematically review backend source code, focusing on authentication, session management, and data access logic. • Use a checklist to look for common vulnerabilities (e.g., improper input validation, hardcoded secrets, use of insecure libraries, potential for injection). • Review frontend code for potential XSS vulnerabilities and improper handling of sensitive data.	• A report is generated documenting all potential security flaws found in the code. • Vulnerabilities are categorized by severity. • Recommendations for remediation are provided.